

Lab program-17:

17. Illustrate the deadlock avoidance concept by simulating Banker's algorithm using C.

Code:

```
#include <stdio.h>

#define MAX_PROCESSES 5
#define MAX_RESOURCES 3

int is_safe();

int available[MAX_RESOURCES] = {3, 3, 2};

int maximum[MAX_PROCESSES][MAX_RESOURCES] =
{
    {7, 5, 3},
    {3, 2, 2},
    {9, 0, 2},
    {2, 2, 2},
    {4, 3, 3}
};

int allocation[MAX_PROCESSES][MAX_RESOURCES] =
{
    {0, 1, 0},
    {2, 0, 0},
    {3, 0, 2},
    {2, 1, 1},
    {0, 0, 2}
};

int request_resources(int process_num, int request[])
{
    // Check if request can be granted
    for (int i = 0; i < MAX_RESOURCES; i++)
    {
        if (request[i] > available[i] ||
            request[i] > (maximum[process_num][i] -
allocation[process_num][i]))
        {
            return 0;
        }
    }

    // Temporary allocation
```

```

    for (int i = 0; i < MAX_RESOURCES; i++)
    {
        available[i] -= request[i];
        allocation[process_num][i] += request[i];
    }

    // Check safe state
    if (is_safe())
    {
        return 1;
    }
    else
    {
        // Rollback
        for (int i = 0; i < MAX_RESOURCES; i++)
        {
            available[i] += request[i];
            allocation[process_num][i] -= request[i];
        }

        return 0;
    }
}

int is_safe()
{
    int work[MAX_RESOURCES];
    int finish[MAX_PROCESSES] = {0};

    for (int i = 0; i < MAX_RESOURCES; i++)
    {
        work[i] = available[i];
    }

    int count = 0;

    while (count < MAX_PROCESSES)
    {
        int found = 0;

        for (int i = 0; i < MAX_PROCESSES; i++)
        {
            if (finish[i] == 0)
            {
                int j;

                for (j = 0; j < MAX_RESOURCES; j++)
                {

```

```

        if ((maximum[i][j] - allocation[i][j]) > work[j])
            break;
    }

    if (j == MAX_RESOURCES)
    {
        for (int k = 0; k < MAX_RESOURCES; k++)
        {
            work[k] += allocation[i][k];
        }

        finish[i] = 1;
        found = 1;
        count++;
    }
}

if (found == 0)
{
    return 0;
}

return 1;
}

int main()
{
    int process_num;
    int request[MAX_RESOURCES];

    printf("Enter Process Number (0-4): ");
    scanf("%d", &process_num);

    printf("Enter Resource Request (e.g. 0 1 0): ");

    for (int i = 0; i < MAX_RESOURCES; i++)
    {
        scanf("%d", &request[i]);
    }

    if (request_resources(process_num, request))
    {
        printf("\nRequest Granted.\n");
    }
    else
    {

```

```
        printf("\nRequest Denied. System is NOT in a Safe State.\n");  
    }  
  
    return 0;  
}
```

OUTPUT:

```
● PS C:\Users\SMILE\Documents\OS\Lab programs> ./lab_17.exe  
Enter Process Number (0-4): 3  
Enter Resource Request (e.g. 0 1 0): 0 1 0  
  
Request Granted.
```